DynamiX Device Reference

Checked against the live device registry

October 8, 2026 · kernel figures drawn by `make_figures.py` from the app's own kernel code

How to read this document

A *device* is one processing tool in a layer's chain. **Transforms** compute (they head chains and produce a result bundle); **filters** select from a computed result (list indexes and masks — scrubbing a filter knob redraws without recomputing). Chains are always transforms-then-filters; interleaving is rejected at construction.

Each tool has a display name and a device id. The device browser lists tools by scientific family and, like the chain strip, shows a tool's display name with its id in grey; this reference names a tool the same way: WTMM 2D · `wtmm2d`. Saved chains, rack files, layer names, cache keys and provenance record the device id. Each tool declares the data it consumes and produces, and one rule derived from those declarations says whether a step applies (§2). The tables of families, contracts, applicability and output rows are written by `make_tables.py` from the tools' own declarations, as the kernel figures are drawn from the kernel code.

Conventions that hold everywhere:

- **Scales are stored in pixels**, converted only for display.
- **The shown level is view state**: the Transport sets it for each layer (§1), and no chain step holds it.
- **The wavelet family belongs to the method**: each tool's menu lists only the kernel class its method admits, under the method's own names.
- **Inert knobs**: a parameter that binds only under one choice of another parameter (e.g. q only for q-family wavelets) is ignored otherwise; where the device declares it, the knob is *greyed out* in the device box while it does not apply (its value is kept).
- **View-only knobs** (marked *view* in the tables) choose what is shown from a result that is already computed and cached — Show, Component, Group, Orientation, Show PC. Switching one redraws at once; it never recomputes.
- **Orientations are** (**angle**, **frame**), the frame being the pixel grid or grid, true or magnetic north (§6.1, Gradient arrows and anisotropy).
- **No-support masking**: pixels with no honest signal support (nodata flats, FFT dust) are NaN in every exponent map.
- Each table lists: parameter (UI label), default, hard range, and meaning. Soft ranges (knob-drag limits) are narrower than hard ranges.
- **A step that does not apply** (a V-chain filter after an analysis without V-chains) stays in its chain with its knobs, drawn inactive with the reason, and the engine skips it. Bypassing a step is the user's own act.

Wavelet normalization: L^1

Every wavelet in the app is normalized in the L^1 (pure-dilation) convention, $\psi_a(x) = a^{-d}\,\psi(x/a)$ in $d = 2$ dimensions, so that at a Hölder singularity the transform scales as $|T_\psi[f](x,a)| \sim a^{h(x)}$ with no offset. The L^2 convention ($a^{-d/2}$, unit energy $\int \psi^2 = 1$ — e.g. the normalization constants A and A_q of Borges et al. 2004, whose transform carries $a^{-1/2}$) shifts every exponent by $d/2$. Turiel's microcanonical formalism uses the same

L^1 dilation $\Psi_r(x) = r^{-d}\Psi(x/r)$ (Turiel et al. 2008, eqs. 6–8). Borges et al.'s own WTMM example agrees in practice: their eq. 41 reads $T \sim a^\alpha$ (their fig. 5 recovers $\alpha = 0.5$), which holds for L^1 scaling, not for the $a^{-1/2}$ of their eq. 10.

- `wtmm2d` **normalizes in the Fourier domain.** Each filter is built as $\hat{\psi}(a\mathbf{k})$ from a smoothing function with $\hat{\theta}(0) = 1$ (unit mass at every scale), times the derivative factor $i\,a\mathbf{k}$: scale enters only through $a\mathbf{k}$, which *is* the L^1 dilation.
- **The Hölder tools normalize in real space** by default, on the pixel grid: positive kernels (`holder_measure`) to unit discrete mass; Mexican hats (`holder_multiaffine`) to exact zero mean, then unit L^1 norm ($\sum|\psi_r| = 1$), at every scale. With *Wavelet space* set to Fourier (§9) they are built as $\hat{\psi}(r\mathbf{k})$ instead, like `wtmm2d`'s filters.
- A constant prefactor that does not depend on the scale (such as A or A_q) cannot change any exponent — it only shifts $\log|T|$ — and the app's own normalization replaces it anyway. Only the *scale dependence* of the normalization matters.
- **Validated on a synthetic cusp** $|x - x_0|^h$: both routes recover the slope h (tests pin it; an L^2 slip would read $h + 1$ in 2-D). The smallest scales are biased by the discretization of both kernel and signal: at a sampled cusp the slope settles above $\sigma \approx 5\,\text{px}$ (`wtmm2d`) and $r \approx 10\,\text{px}$ (the Mexican hats), later for sharp-cutoff kernels ($q_T < 1$).
- **Two different q's.** q_T (the knob *Tsallis q* of the Hölder tools) is the Tsallis/Borges shape parameter of a kernel; q in $Z(q, a)$, $\tau(q)$ and $D(h)$ is the moment order of the partition functions. They are unrelated.

Contents

1 The workspace: datasets, layers, ROIs, derivatives **3**

2 Tool families and data contracts **8**

3 Racks **11**

4 Field preparation **12**
4.1 Noise / dither · `noise` — perturbation . . . 12
4.2 Band select · `band_select` — one band of a stack . . . 12
4.3 Fill · `fill` — the inpainting engine as a field stage . . . 12

5 Routing **13**
5.1 `bus` — sends into one stack . . . 13

6 Primary analyzers **13**
6.1 WTMM 2D · `wtmm2d` — canonical WTMM (Arnéodo) . . . 13
6.2 WTMM 2D, ROI · `wtmm2d_roi` — WTMM over one ROI window . . . 14
6.3 Mallat–Zhong edges · `mz_edges` — dyadic multiscale edges . . . 14
6.4 Wavelet skeleton · `wavelet_skeleton` — medial-axis (modulus-minima) extractor . . . 16
6.5 CDF edges · `cdf_edges` — complex cross-diffusion edges . . . 17
6.6 Perona–Malik edges · `pm_edges` — anisotropic diffusion edges . . . 18

7 Decompositions **18**
7.1 PCA · `pca` — principal components of a multi-band field . . . 18
7.2 Tucker (HOOI / HOSVD) · `tucker_HOOI_HOSVD` — delay-embedded Tucker decomposition (Tucker-SSA) . . . 18
7.3 2-D SSA · `ssa2d` — 2D singular-spectrum analysis . . . 20
7.4 DWT · `dwt2d` — the periodized orthogonal pyramid . . . 21

8 Inpainting and fusion **21**
8.1 Inpaint · `inpaint` — fill declared regions . . . 21

9 Microcanonical MFA **22**
9.1 Microcanonical MFA, measure · `holder_measure` — gradient-measure route . . . 23
9.2 Microcanonical MFA, multiaffine · `holder_multiaffine` — wavelet-projection route . . . 23
9.3 Band reconstruction: the `band` and `band_mask` outputs . . . 25

10 Topology **26**
10.1 Chain topology · `chain_topology` (Transform) . . . 26
10.2 Min V-chains per H-line · `min_vchains` (Filter) . . . 26

11 Selection: extrema **27**

12 Selection: V-chains **27**
12.1 Seam classifier · `chain_classify` . . . 27

13 Groups, points, Dev **28**

14 Planned: q-generalized wavelets **28**

1 The workspace: datasets, layers, ROIs, derivatives

Datasets. Every opened file is a *dataset*: one row in the layer tree, and that row *is* its raw layer (its H hides the raster; right-click → Remove dataset takes the dataset and everything on it). A plain image (PNG/JPEG) opens as one luminance band; a multiband GeoTIFF / netCDF / HDF stack opens as (n_y, n_x, n_c).

Children. A tool dropped on a dataset never replaces the data: it spawns a *child* layer holding that tool, nested under the dataset. A child re-runs its chain on its parent's data whenever that changes. Right-click on a result row: *Delete layer* (its children move up to its parent) or *Delete layer and children*; shift-click or Cmd-click (Ctrl on Windows) selects several rows for one Delete. **H never cascades**: each row hides only its own drawing (the dataset's H hides the dataset raster, not the results over it).

ROIs. A saved region of interest is stored in whole *file* pixels and shown as a row `dataset @A` under its dataset. Selecting the ROI row makes it the region the next tool runs on; results (`tool @A`) nest under it. The ROI row's H hides only its outline. An ROI cannot be deleted while results use it (delete them first). Analyzers read their own declared margin around the ROI; decompositions (`pca`, `tucker_HOOI_HOSVD`, `ssa2d`) analyse the ROI itself (margin 0: a margin would mix outside data into the statistic).

Output rows. A result whose tool produces several things carries a collapsed *Outputs* group under its row: one row per output, each with its own H (the output-rows table below lists every tool's rows). Raster outputs share the canvas one at a time: showing one hides the others, and hiding the one shown puts back what the tool's `show` shows by default (the data, or an MFA tool's h map). They follow the tool's `show` knob, and switching between them re-uses what was computed. The *edges* row hides the maxima dots and gradient arrows (and the H-lines in the Vector and Geo views); the result keeps its maxima, so filters, the spectrum and the anisotropy window still read them. The layer's own H hides everything the result draws. An output that needs its own computation (the `mz_edges` reconstruction, an MFA tool's band, one level of a per-scale output, §1) runs the first time it is shown, with progress and Stop, and stays cached; Freeze (the row's F) keeps it.

Output rows: the rows each tool carries under its result.

Tool	Output	Row	Kind	Computed	Per level
WTMM 2D · `wtmm2d`	`modulus`	modulus	raster	when shown	yes
	`argument`	argument	raster	when shown	yes
WTMM 2D, ROI · `wtmm2d_roi`	`modulus`	modulus	raster	when shown	yes
	`argument`	argument	raster	when shown	yes
Mallat–Zhong edges · `mz_edges`	`edges`	edges	vector	with the analysis	
	`coarse`	coarse	raster	when shown	

Tool	Output	Row	Kind	Computed	Per level
	`thumbnail`	thumbnail	raster	when shown	
	`recon`	recon (edges + coarse)	raster	when shown	
	`recon_edges_only`	recon (edges only)	raster	when shown	
	`residual`	residual	raster	when shown	
	`modulus`	modulus	raster	when shown	yes
	`argument`	argument	raster	when shown	yes
CDF edges · `cdf_edges`	`edges`	edges	vector	with the analysis	
	`filtered`	filtered	raster	with the analysis	
	`edge_channel`	edge channel	raster	with the analysis	
	`re_scale`	Re at σ	raster	when shown	yes
	`im_scale`	Im at σ	raster	when shown	yes
Perona–Malik edges · `pm_edges`	`edges`	edges	vector	with the analysis	
	`filtered`	filtered	raster	with the analysis	
	`filtered_scale`	filtered at σ	raster	when shown	yes
Microcanonical MFA, measure · `holder_measure`	`band`	band reconstruction	raster	when shown	
	`band_mask`	band mask	raster	when shown	
Microcanonical MFA, multi-affine · `holder_multiaffine`	`band`	band reconstruction	raster	when shown	
	`band_mask`	band mask	raster	when shown	
DWT · `dwt2d`	`approximation`	approximation	raster	when shown	yes
	`horizontal`	horizontal	raster	when shown	yes
	`vertical`	vertical	raster	when shown	yes
	`diagonal`	diagonal	raster	when shown	yes
Inpaint · `inpaint`	`residual`	residual	raster	when shown	
	`heldout`	held-out error	raster	when shown	

Progress is live: every compute reports one increasing fraction from inside its own loops.

The right panel shows what applies to the selected row: the colours, widths and arrows of the WTMM overlays appear with a result that has maxima or chains, *Reconstruct…* with an h map, colormap, stretch and hillshade with a single-band display (a multiband display uses the Composite section), and the view-specific sections in the Vector and Geo views.

Derivative datasets (Fork)

A *child* keeps re-processing its parent. A **derivative** is forked once from what a result shows and from then on is an *original raw dataset*: nothing links it back, and a tool dropped on it spawns a child that analyses the derivative's own pixels.

Where	result row → right-click → *Fork derivative dataset…*; or the *Fork derivative…* button of the Components window

Raster	a checklist of every raster the result holds: the one on screen first (after its view knobs — e.g. Group "1", Show residual = data − C1), then the components / PCs, recon and residual per input band of a multiband field, filtered, h(x)…; every checked raster becomes a band (n checked → an n-band dataset)
Vector	the extrema and maxima lines the result shows (after its filters)
Both	raster and vector go in one file
Placement	a dataset row of its own, or nested inside the dataset it came from (removing that dataset lifts the derivative to the top)
Keep	*Save to a file…* (a Save-As dialog; permanent), or *Temporary*: a file in this session's scratch folder, left out of a saved project (a notice names it) until *Save derivative as…* on its row writes it where you choose
Grid	the result's own grid, never resampled: the whole field, or the ROI window for an ROI result
Georeference	the parent's CRS rides over (with the grid's axes); the parent file's layout (its source path, window, full size) does not
File	a RasterField `.npz` (values, axes, frame, units, provenance) — it opens like any `.npz`; vectors ride under `vec_*` keys
History	`provenance["derived"]`: parent path and SHA-256, the layer, its whole chain with every parameter (view knobs included), the ROI window, the date; `provenance["bands"]` names the bands

A derivative that carries vectors gets `derived_vectors` (a loader the shell places, in the browser's Dev group) as its first step: its frozen extrema and maxima lines draw like any result's, and the extrema and chain filters can be dropped after it.

The Components window

Right panel → Decomposition → *Components…*, on a `ssa2d` or `tucker_HOOI_HOSVD` result:

- **thumbnails** — the eigenarrays (`ssa2d`: the $L_r \times L_c$ window patterns) or the component images, each labelled with its share;
- **the shares** on a log scale (group members in amber) — a slowly decaying floor is noise;
- **the w-correlation matrix** (`ssa2d`; black 0 to white 1) — separable components are nearly w-orthogonal, so groups show as blocks on the diagonal; hovering reads $w(C_i, C_j)$.

Clicking thumbnails picks the **group**: click one, Cmd-click (Ctrl) to add or remove, shift-click for a range, *All* for every one. The pick is written into the tool's Group knob (and the window follows the knob), so the canvas redraws from the cache.

Multidimensional rasters

Import. Opening a multi-grid container (netCDF, HDF5, HDF4) shows an import dialog: the image grids grouped by sensor and resolution with a checkbox per band, and the remaining grids (geolocation lattices, calibration tables, supplements) in a collapsed *Ancillary grids* group, each importable on its own. Each ticked group becomes one multiband dataset (n_y, n_x, n_c). Grids of different resolution are separate datasets: ASTER's VNIR (15 m), SWIR (30 m) and TIR (90 m) arrive as three. HDF4 grids are named by their swath group (`VNIR_Band3N/ImageData`), since ASTER names every band's array `ImageData`. A dataset is its file together with its band list, so two groups imported from one file are two datasets, each with its own row, cache and saved state.

Band rows. A multiband dataset lists its bands under a collapsed *Bands (n)* row. Each band row is a child layer whose chain starts with `band_select`: selecting it shows that band, and a tool dropped on it runs on that band alone. *Remove band from project* (right-click, or Delete) takes the band out of the dataset together with its row and any results under it; the file is untouched, and the last band stays. The removal changes the dataset's identity, so a result computed on the fuller stack is recomputed.

Composite. A multiband dataset draws as an RGB composite. The *Composite* section of the right panel lists the bands with a channel (R, G, B or none; each channel takes one band) and Solo / Mute buttons: mute drops a band's channel; one solo shows that band alone in grey; several solos keep each in its channel. Its *Stretch* applies one type to all channels, each computed from its own band's values:

Stretch	Mapping to $[0, 1]$
linear (min–max)	minimum to maximum (normalisation)
percent clip	the p-th to $(100 - p)$-th percentiles, clipped ($p = 2$ is ENVI's linear 2 %)
std-dev	mean $\pm k\sigma$, clipped
log	$\log(1 + v - \min v)$, then minimum to maximum
histogram equalize	each value's rank among the finite values
bipolar	$-m$ to $+m$, m the $(100 - p)$-th percentile of $\lvert v \rvert$: zero maps to mid-scale, for signed data (principal components, residuals)

The single-band Display stretch offers the same six types. Stretches are display only; analysis reads the band values themselves.

What a field stage shows. A chain that ends on a field stage shows the field it produces: `noise` alone shows the noised field (a stack through the composite), a band row its band, a bus its stack. Bypassing the stage shows the raw data again, and *Fork derivative dataset…* takes the produced field, one band per checkbox.

Buses: routing bands into one stack

Multi-band tools (`pca`, `tucker_HOOI_HOSVD`) take a stack (n_y, n_x, n). A **bus** assembles one from the bands of any datasets on the same grid. It heads a child layer's rack (`bus`), and a tool dropped after it analyses the bus's stack. Right-click a dataset row → *Route bands to a new bus…*:

- **Sends** are picked per band from every loaded dataset; a dataset on another grid is listed greyed out with the reason. A send references the band in its file (the file path and the band's grid id or index); the bus reads the band when it computes, over the window it needs.
- **Channel order** (drag to reorder) is the band axis the tool receives. `pca` and `tucker_HOOI_HOSVD` give the same decomposition in any order, with the per-band loadings in that order; band math and the composite's default channels depend on it.
- **Same grid**: every send has the bus's shape, pixel step, georeference anchor and CRS. Combining grids of different resolution requires resampling, which the bus does not do.

- **Windows**: on an ROI the bus reads each band over the ROI's processing window, reflect-padded at the file edges exactly as the dataset itself is.
- **Returns**: the results of the tool after the bus are ordinary result rows; forking one makes it a dataset of its own.
- *Edit bus sends…* (right-click the bus row) re-opens the routing; a routing already computed is answered from the cache.

Live band stacks. Select two or more layers (shift- or Cmd-click), right-click → *Build band stack from N selected layers*: a bus whose sends are the rasters those layers show (a reconstruction, a noised band, an h map), referenced live. Each send carries a fingerprint of its layer's chain; editing a referenced layer changes the fingerprint, and the stack recomputes when next shown. Referenced layers are computed first when needed, and a deleted one is reported by name. The layers share the stack's grid (an ROI result has its own grid and is refused). Layer sends and file sends mix in one bus.

The shown level: the Transport

The Transport under the canvas (a slider, play, and a reading of the scale in pixels and in the dataset's units) sets the level the active layer shows. The level is view state: each layer keeps its own, saved with the project, and scrubbing it leaves the chain as it is. After the last Transform that applies, when the analysis holds an extrema stack of more than one level, the engine takes the shown level out of the stack and then runs the filters, so a filter knob redraws one level. The V-chains stay whole: the 3-D scale-space view lifts them through every scale, and the anisotropy window reads every level. The Transport works on every multi-scale layer, the *WTMM + Hölder* rack's included, and the vector (3-D) view draws a layer's extrema at the level that layer shows. Bands stay rows of their dataset; the Transport ranges over scales only.

Each inspector window has a Transport of its own. With its *M* button on (the default) it follows the main Transport; with it off the inspector holds a level of its own. Two views of one layer at two levels resolve the same cached analysis, each at its level.

`scale_select` is superseded. It stays registered, in the browser's Dev group. Of the built-in racks only *WTMM standard*, the default chain, holds it. Opening a project whose chain holds it sets the layer's level from its `scale_idx` once; the step stays in the chain, drawn inactive with the reason "the scale is set by the Transport".

Per-scale outputs and levels marked for figures

The multi-scale analyzers carry one raster row per scale-space output, off by default (*per level* in the output-rows table):

- WTMM 2D · `wtmm2d` and WTMM 2D, ROI · `wtmm2d_roi`: `modulus` and `argument`, the wavelet transform's modulus and argument at one scale;
- Mallat–Zhong edges · `mz_edges`: `modulus` and `argument` at one dyadic level;
- CDF edges · `cdf_edges`: `re_scale` and `im_scale`, the Re and Im snapshots at σ_k;
- Perona–Malik edges · `pm_edges`: `filtered_scale`, the diffused raster at σ_k.

A row that is on shows the level the Transport is on and reads that level's scale in pixels; a row that is off computes nothing, and the levels it computed while on stay cached until the bounds below evict them. Turning a row on is a choice for that layer alone. Each level is its own cache entry. WTMM 2D and the Mallat–Zhong edges compute the one level asked for (one FFT pair, one filter cascade); the CDF and Perona–Malik evolutions are sequential, so the first request runs the evolution once and keeps every snapshot as one entry. Each output runs the tool's own per-scale routine with the analysis's knobs (WTMM 2D: its dither and a^η lift as the analysis applied them); with the `nms` detector the maxima of WTMM 2D's level-k modulus are the stored level-k extrema. A WTMM 2D, ROI · `wtmm2d_roi` layer made by *Save + run WTMM* draws its rows on the ROI's own grid; a result run on an ROI row (`tool @A`) computes no lazy outputs.

Marking a level for figures. Right-click a per-level row → *Mark level n for figures* (n counts from 1 at the finest level; *Unmark* on a marked level). The row reads *marked*, and the marks are saved with the layer. A

marked level is pinned in the cache while its row is on, so eviction passes it by; unmarking it, or turning the row off, releases it. *File → Export Derived Raster (.npz)…* writes the level on show; its provenance names the output (`<device id>/<output>`), the level and the marked levels (counted from 0), and the status line lists the marked levels.

Memory. The cache holds at most 64 unpinned entries. The raster outputs also share a byte budget, 2 GB by default, set by `output_cache_bytes` in the settings file (§3 gives its folder); a 4096^2 float32 level is about 67 MB. Past the budget the least recently used unpinned output levels go first. Fields, stacks and analysis results stay under the entry bound alone.

2 Tool families and data contracts

Families. The device browser lists tools by scientific family (the families table), and a tool in several families is listed under each. The *Dev* group, collapsed, holds the parameter-schema stubs, the superseded tools (kept registered so that saved projects open and resolve) and the steps the shell places itself. A tool that declares no family is listed by its id under *Other*. The last group, *Racks*, lists the built-in racks and the user's own (§3).

Families: the tools the device browser lists under each family.

Family	Tools (display name · device id)
Data preparation	Noise / dither · `noise`; Fill · `fill`; Band select · `band_select`
Edge detection	WTMM 2D · `wtmm2d`; WTMM 2D, ROI · `wtmm2d_roi`; Mallat–Zhong edges · `mz_edges`; CDF edges · `cdf_edges`; Perona–Malik edges · `pm_edges`
Regularity	WTMM 2D · `wtmm2d`; WTMM 2D, ROI · `wtmm2d_roi`; Mallat–Zhong edges · `mz_edges`; Microcanonical MFA, measure · `holder_measure`; Microcanonical MFA, multiaffine · `holder_multiaffine`
Skeletons	Wavelet skeleton · `wavelet_skeleton`
Decomposition	PCA · `pca`; Tucker (HOOI / HOSVD) · `tucker_HOOI_HOSVD`; 2-D SSA · `ssa2d`; DWT · `dwt2d`
Reconstruction	Mallat–Zhong edges · `mz_edges`; Microcanonical MFA, measure · `holder_measure`; Microcanonical MFA, multiaffine · `holder_multiaffine`; Tucker (HOOI / HOSVD) · `tucker_HOOI_HOSVD`; 2-D SSA · `ssa2d`; Inpaint · `inpaint`
Topology	Chain topology · `chain_topology`
Selection: extrema	Orientation wedge · `orientation_wedge`; Modulus threshold · `modulus_threshold`; H-line length · `hline_length`; H-line modulus · `hline_modulus`; H-line Hölder · `hline_holder`
Selection: V-chains	Min V-chains per H-line · `min_vchains`; Chain Hölder · `chain_holder`; Chain modulus · `chain_modulus`; Chain length · `chain_length`; Seam classifier · `chain_classify`; Group filter · `group_filter`
Points	Back-project points · `backproject`
Dev (collapsed)	`bus`; `holder_map`; `band_recon`; Band reconstruction, measure · `band_recon_measure`; Band reconstruction, multiaffine · `band_recon_multiaffine`; `tucker_havok`; Scale · `scale_select`; `group_paint`; `derived_vectors`; `stub_wavelet`; `stub_holder`; `stub_wedge`

Contracts. Devices pass named kinds of data, the contracts, and every tool declares the ones it consumes and the ones it produces (the declarations table). In a declaration "or" separates alternatives (Tucker takes a Field or a Stack) and "+" joins contracts needed together. *What it received* passes the step's input through: `noise` keeps a field a field and a stack a stack, and `chain_topology` adds the topology to the analysis it reads. A declaration marked † depends on the parameters; the table gives it at the defaults, then each choice that changes it. A `bus` produces a Field from one send and a Stack from several.

Contracts: the named kinds of data and the result keys each one covers.

Contract	Meaning	Result keys
Field	a single-band field	the `RasterField` itself
Stack	a band stack	the `RasterField` itself
Points	a point set	the `PointSet` itself
ExtremaStack	an extrema stack	`extrema`, `scales`, `_shape`
PointAlpha	a per-point α	`extrema[i]["alpha"]`
VChains	V-chains	`chains`
Topology	the H/V topology	`topology`, `_topology_vids`
HMap	an h-map	`h_map`, `r2_map`
Components	decomposition components	`raster_out`

Declarations: what each tool consumes and produces (T: Transform, F: Filter; †: depends on the parameters).

Tool		Consumes	Produces
Noise / dither · `noise`	T	Field or Stack	what it received
Fill · `fill`	T	Field or Stack	what it received
Band select · `band_select`	T	Stack or Field	Field
`bus`	T	Field or Stack	Stack†
WTMM 2D · `wtmm2d`	T	Field	ExtremaStack + VChains
WTMM 2D, ROI · `wtmm2d_roi`	T	Field or Stack	ExtremaStack + VChains
Mallat–Zhong edges · `mz_edges`	T	Field	ExtremaStack + PointAlpha†; `algorithm` = `printed`: ExtremaStack
Wavelet skeleton · `wavelet_skeleton`	T	Field	ExtremaStack
CDF edges · `cdf_edges`	T	Field	ExtremaStack
Perona–Malik edges · `pm_edges`	T	Field	ExtremaStack
`holder_map`	T	Field	HMap
Microcanonical MFA, measure · `holder_measure`	T	Field	HMap
Microcanonical MFA, multiaffine · `holder_multiaffine`	T	Field	HMap
`band_recon`	T	Field	HMap
Band reconstruction, measure · `band_recon_measure`	T	Field	HMap
Band reconstruction, multiaffine · `band_recon_multiaffine`	T	Field	HMap
PCA · `pca`	T	Stack	Components
`tucker_havok`	T	Field or Stack	Components
Tucker (HOOI / HOSVD) · `tucker_HOOI_HOSVD`	T	Field or Stack	Components
2-D SSA · `ssa2d`	T	Field	Components
DWT · `dwt2d`	T	Field	Components
Inpaint · `inpaint`	T	Field or Stack	Components
Chain topology · `chain_topology`	T	ExtremaStack + VChains	what it received + Topology
Min V-chains per H-line · `min_vchains`	F	Topology	—
Scale · `scale_select`	F	ExtremaStack	—
Orientation wedge · `orientation_wedge`	F	ExtremaStack	—

Tool		Consumes	Produces
Modulus threshold · `modulus_threshold`	F	ExtremaStack	—
H-line length · `hline_length`	F	ExtremaStack	—
H-line modulus · `hline_modulus`	F	ExtremaStack†; `mode` = `sup_all_scales`: ExtremaStack + VChains	—
H-line Hölder · `hline_holder`	F	ExtremaStack + PointAlpha	—
Chain Hölder · `chain_holder`	F	VChains	—
Chain modulus · `chain_modulus`	F	VChains	—
Chain length · `chain_length`	F	VChains	—
Seam classifier · `chain_classify`	F	VChains	—
`group_paint`	F	VChains	—
Group filter · `group_filter`	F	VChains	—
Back-project points · `backproject`	T	Points	Points
`derived_vectors`	T	Field or Stack	ExtremaStack + VChains

The rule. Before the first step the layer's input is available: a Field for a single-band raster, a Stack for an (n_y, n_x, n_c) raster with $n_c > 1$, Points for a point set. A band row's `band_select` turns its dataset's Stack into a Field. Then, step by step:

- a Transform applies when one of its alternatives is available, and its products then replace the available set;
- a Filter applies when one of its alternatives is available, and leaves the set as it is;
- a step that does not apply leaves the set as it is.

The engine skips a step that does not apply, so the step is absent from the dependency set and from the cache keys, and the last Transform that applies is the analysis the layer's output rows come from. The chain strip draws the step inactive with the reason, which names the missing contract, the tools that produce it and the current analysis: "needs V-chains: WTMM 2D and WTMM 2D, ROI produce it; the current analysis (CDF edges) does not". The step keeps its place and its knobs, and it runs again as soon as its input is there. A tool that declares no consumes always applies, and a Transform that declares no products counts as producing every contract, so a tool from outside this build keeps working.

Where the rule is used.

- **Browser.** A tool that cannot apply as the next step on the active layer is greyed, with the reason in its tooltip. A greyed row still drags, and the step lands drawn inactive. A tool that reads a Field or a Stack stays available on an analysed layer, because dropping it there forks a sibling.
- **Chain strip.** Each step that does not apply is drawn inactive with its reason.
- **Fork.** A primary analyzer reads a Field or a Stack and produces neither: the edge, regularity, skeleton and decomposition tools. Dropped onto a layer headed by a different one, it forks a sibling layer; dropped onto a dataset row, it spawns a child. The sibling begins with the parent's leading routing steps (a band row's `band_select`, a `bus` with its sends), so it analyses the same field.
- **Shipping.** A filter on the parent moves to the sibling, with its knobs, when it applies after the sibling's analysis and does not apply after the parent's.
- **Racks.** A rack is greyed where its first step cannot apply, and the rack editor runs the rule along the rack (§3).

Which tool applies after which. The applicability table is the rule run for every selection tool after every analysis. The extrema filters apply after each tool that produces an extrema stack. The V-chain filters, Chain topology · `chain_topology` and `hline_modulus` in mode `sup_all_scales` apply after the WTMM tools,

which produce V-chains. Min V-chains per H-line · `min_vchains` applies after Chain topology, and H-line Hölder · `hline_holder` after the Mallat–Zhong edges of the authors' algorithm, whose analysis stamps a per-point α. A derivative dataset's frozen vectors (`derived_vectors`) carry an extrema stack and V-chains, the contracts of a WTMM 2D analysis, so the same tools apply after them.

Applicability: • where a selection tool applies after an analysis (the contracts rule, §2).

applies after →	WTMM 2D	WTMM 2D, ROI	Mallat–Zhong edges	Mallat–Zhong edges, printed	Wavelet skeleton	CDF edges	Perona–Malik edges	Microcanonical MFA, measure	Microcanonical MFA, multiaffine	PCA	Tucker (HOOI / HOSVD)	2-D SSA	DWT	Inpaint	WTMM 2D + Chain topology	WTMM 2D, ROI + Chain topology
Chain topology · `chain_topology`	•	•													•	•
Min V-chains per H-line · `min_vchains`															•	•
Orientation wedge · `orientation_wedge`	•	•	•	•	•	•	•								•	•
Modulus threshold · `modulus_threshold`	•	•	•	•	•	•	•								•	•
H-line length · `hline_length`	•	•	•	•	•	•	•								•	•
H-line modulus · `hline_modulus`	•	•	•	•	•	•	•								•	•
H-line modulus · `hline_modulus`, `mode` = `sup_all_scales`	•	•													•	•
H-line Hölder · `hline_holder`			•													
Chain Hölder · `chain_holder`	•	•													•	•
Chain modulus · `chain_modulus`	•	•													•	•
Chain length · `chain_length`	•	•													•	•
Seam classifier · `chain_classify`	•	•													•	•
Group filter · `group_filter`	•	•													•	•

3 Racks

A *rack* is a named chain to drop onto a layer. The browser's *Racks* group lists the built-in racks and the user's own; double-clicking one opens it in the rack editor.

Built-in	read-only: *WTMM standard* (the default chain), *WTMM + Hölder*, *M–Z edges*, *CDF edges*, *PM edges* and *Wavelet skeleton*; *Duplicate to edit* in the editor makes a user copy
Save	*Save chain as rack…* (right-click a result or band row) or *Save as rack…* (the chain strip) records every step with all its knobs, view-only ones included, and its bypass state, under the name you give. A band row's own `band_select` and a `bus` with its sends belong to their layer and stay out
Store	one JSON file per rack, named after the rack, in the `racks` folder beside the settings file: `~/Library/Application Support/GeoDynamix/racks/` on macOS, `%APPDATA%\GeoDynamix\racks\` on Windows; with `DYNAMIX_SETTINGS_PATH` set, beside the settings file it names

File	{"format": "geodynamix-rack", "schema": 1, "name", "description", "steps": [{"device", "params", "bypassed"}], "created", "app_version"}, with device ids and parameter names as a saved chain stores them
Editor	*Racks* → *Rack Editor…*: the family tree on the left and the rack's steps on the right; drag a tool in to add it and drag a step to reorder; the selected step's knobs; *Save*, *Save as…*, *Delete* (user racks) and *Duplicate to edit*
Validation	live while editing: the order must be transforms-then-filters, and the contracts rule (§2) runs along the rack from the input chosen under *Validate from a*: a single-band field, a band stack or a point set
Greyed	a rack whose first step cannot apply on the active layer is greyed with the reason. A rack file that names a device or a parameter this build lacks, or that cannot be read, is listed greyed with the reason, and the other racks load
Drop	split and merge: the rack's Transforms go after the chain's Transforms and its Filters after the chain's Filters, so the order stays legal. A rack Transform whose tool already runs in the chain is left out. A rack headed by a different primary analyzer forks a sibling, as dropping that analyzer does

4 Field preparation

4.1 Noise / dither · noise — perturbation

Adds controlled noise to the field before analysis. Its main use is *dithering* quantized data ($\pm\frac{1}{2}$ LSB) so that quantization plateaus do not read as zero-gradient flats.

Parameter	Default	Range	Meaning
dist (Distribution)	uniform	uniform / gaussian	noise law
amplitude	0	$[0, 10^9]$	0 = auto $\frac{1}{2}$ LSB; uniform: half-width; gaussian: σ
seed	0	$[0, 2^{31}]$	RNG seed (deterministic reruns)
sign (Apply)	add	add / subtract	subtract undoes a previous add exactly

4.2 Band select · band_select — one band of a stack

Slices one band out of a multiband field as the working field; grid, frame and CRS are unchanged. Band rows start with it and name their band, so removing another band from the dataset leaves each row on its own band.

Parameter	Default	Range	Meaning
band (Band)	1	$[1, 256]$	the band, 1-based (a band row selects by name)

4.3 Fill · fill — the inpainting engine as a field stage

Fills the declared regions of the host band (or of target_band on a stack) before the analyzer, with inpaint's engine and knobs (below) but no held-out blocks and no Show. The clone carries filled_fraction, fill_route, fill_kind and the run's stamps in its provenance, and every analyzer result downstream carries them as _fill, so the strip and the Band dialog say that analysis inside the region reads a reconstruction. With mask_source = region and no region drawn it passes the field through unchanged (the one no-op decidable from the parameters alone); a data-decided empty mask runs and records filled_fraction = 0.

5 Routing

5.1 `bus` — sends into one stack

The head of a bus layer's rack (§1, Buses). It has no knobs: the routing dialog writes its send list. Its output is the stack (n_y, n_x, n) in channel order, or one band for one send, on the incoming field's grid (the whole dataset, or an ROI's processing window). The shell places it, and the browser lists it in the Dev group.

6 Primary analyzers

Each primary analyzer is its own representation of the field: it reads a single-band field (or a band stack) and produces an analysis. Dropping a different primary analyzer onto an analysed layer forks a sibling layer, and the filters that apply only after the new analyzer move with it (§2).

6.1 WTMM 2D · `wtmm2d` — canonical WTMM (Arnéodo)

The canonical multifractal machinery: a continuous wavelet transform over a geometric scale grid, modulus *maxima* detected per scale (along-gradient non-maxima suppression, or xsmurf's `follow`), maxima chained across scales into maxima lines, feeding partition functions $\rightarrow \tau(q) \rightarrow$ Legendre $D(h)$. The maxima are dense by design — they are the statistical support, not a lineament map (use the filters, or the other analyzers, for sparse features). Legendre spectra are concave hulls by construction; the microcanonical tools (§9) measure $D(h)$ directly — together they diagnose non-concavity.

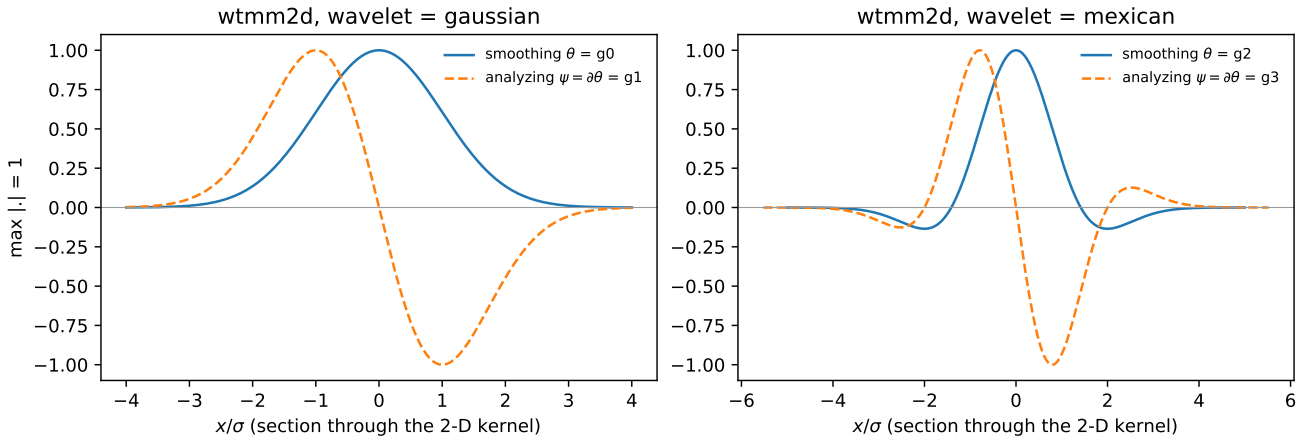


Figure 1: `wtmm2d`: the smoothing function θ and the analyzing wavelet $\psi = \partial\theta$ on the same axes (sections through the 2-D kernels, x in units of the smoothing σ). The wavelet is the derivative of the smoothing function, so the modulus maxima of ψ are the edges of θ-smoothed data.

Parameter	Default	Range	Meaning
`n_oct` (Octaves)	3	$[1, 12]$	scale range: a spans `n_oct` octaves above $a_{\min}$
`n_voice`	4	$[1, 64]$	scales per octave (grid density)
`a_min` ($a_{\min}$)	1.0	$[0.01, 64]$ px	finest analyzed scale
`wavelet`	mexican	mexican / gaussian	CWT kernel (Mexican hat = LoG; gaussian = gradient pair)
`dither`	off	bool	$\pm\frac{1}{2}$ LSB dither for quantized rasters
`fracint_alpha` (η)	1.0	$[-6, 6]$	per-scale a^η lift (pseudo fractional integration; shifts effective h by η)
`detector` (Detector)	nms	nms / follow	maxima detection: non-maxima suppression along the gradient, or xsmurf's `follow` (contour following of the zero-crossings of κ)

`interpolate`	off	bool	subpixel refinement of each maximum along its gradient (`follow` always refines)
`min_chain_len`	2	$[1, 1024]$	discard maxima lines spanning fewer scales
`smooth`	on	bool	smooth maxima-line traces
`thresh`	0.001	$[0, 1]$	modulus floor for maxima detection (fraction)
`dist2_max`	50	$[0, 10^6]$ px^2	max squared jump when chaining maxima across scales
`box_ratio`	1.0	$[0.001, 64]$	chaining search-box anisotropy
`similitude`	0.8	$[0, 1]$	cross-scale similarity acceptance for chaining
`show` (*view*)	edges	3 outputs	`edges` (the edge stack), or the `modulus` or `argument` at the shown level (the Outputs rows, §1)

Gradient arrows and anisotropy

The argument $\mathcal{A} = \arg T_\psi$ of each maximum points uphill in the pixel frame: 0° toward increasing column, +90° toward increasing row (clockwise on screen). **Gradient arrows** (Display section: off / on WTMMM / on all maxima) draw one arrow per point at the current scale, half the scale long (at most 12 px) and scaled by modulus. The WTMMM are the maxima of the modulus along each maxima line (xsmurf's `ssm`).

Anisotropy (Spectrum section) plots the WTMMM statistics of Arnéodo, Decoster and Roux (2000, §6): the density $P_a(\mathcal{A})$ at several scales, flat for an isotropic field; the WTMMM of one scale in the gradient plane (T_{ψ_1}, T_{ψ_2}), round for an isotropic field; and the density of $\log_2 \mathcal{M}$ in four axial sectors of $\mathcal{A}$ (mod 180°, centred on 0°, 45°, 90°, 135°), whose shapes coincide when $\mathcal{M}$ and $\mathcal{A}$ are independent.

Frames. Angles are shown in the pixel frame or, for a georeferenced dataset, as azimuths: from grid north for a projected CRS (from the directions its axes run), from true north by adding the meridian convergence at each point, and from true north directly on a longitude/latitude grid. A dataset without a CRS (a sensor swath, a pixel grid) offers the pixel frame only.

6.2 WTMM 2D, ROI · `wtmm2d_roi` — WTMM over one ROI window

The parent's WTMM re-pointed at a native-resolution window: the knobs of `wtmm2d` plus the window itself. Used by the overview drill-down: draw an ROI on the decimated whole-extent, compute native.

Parameter	Default	Range	Meaning
`roi_row`, `roi_col`	0	px	window origin (native pixels)
`roi_h`, `roi_w`	512	≥ 8 px	window size
`boundary`	auto	auto / reflective	boundary handling at the window edge

6.3 Mallat–Zhong edges · `mz_edges` — dyadic multiscale edges

Mallat and Zhong's multiscale edges ("Characterization of Signals from Multiscale Edges", 1992), computed with the authors' own algorithm: LastWave's `dwtrans2d` package (Bacry, Mallat, Zhong, Hwang et al.), ported to numba and matching its C bit for bit on square fields. The dyadic wavelet transform is computed à trous by direct FIR convolution with the paper's Table I filters ($h = \{\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}\}$ and a two-tap derivative g). The edges at scale 2^l px are `extrema2`'s gradient-modulus maxima: the gradient direction snapped to the row or column axis, a neighbour whose gradient points the opposite way counted lower, and a gap-filling pass at the finest scale; they are linked into per-scale H-lines (`line_id`). Stored moduli carry LastWave's normalization (factors close to $1/\lambda_l$). `Border` *mirror* runs the whole algorithm on the field mirrored to twice its size, for rasters whose edges do not wrap; *periodic* is LastWave's own mode. Two defects of the C are corrected: non-square fields and a single level ($J = 1$). `Algorithm` *printed* selects the paper's printed algorithm as the research port implements it (Fourier-domain filters on the mirrored torus), kept for comparison. Fields with nodata pixels are refused, and numba is required.

Where the edges are drawn. In the authors' transform a sample at index j describes position $j - \frac{1}{2}$, so maxima and the coarse channel from the second level up are drawn half a pixel west and north of their index. At the finest level the two gradient components sit half a pixel apart (W_x at $(j - \frac{1}{2}, i)$, W_y at $(j, i - \frac{1}{2})$), as in the authors' convention; `Co-locate level 1` averages each onto $(j - \frac{1}{2}, i - \frac{1}{2})$ for detection only, and the stored values the reconstruction uses are unchanged.

Reconstruction from the edges. The Outputs rows show *coarse* (the coarse channel S_{2^J} in the data's units), *thumbnail* (S_{2^J} sampled every 2^J pixels, each block centred on its sample), *recon (edges + coarse)*, *recon (edges only)* (with $S = 0$) and *residual* (the data minus the reconstruction). The reconstruction is LastWave's `e2recons`: an initial pass, then alternating projections. Each iteration analyses the current image, restores the stored values at the maxima and spreads each correction between neighbouring maxima along rows (W_x) and columns (W_y) as a sinh interpolation with decay $e^{-\kappa/2^l}$ per sample (row and column ends pinned to zero, as in the C), optionally clips the modulus to decrease away from each maximum, and synthesizes with the coarse channel held fixed. $\kappa = 1$ is the paper's constant; LastWave uses $\kappa = 2 \ln 5.8 \approx 3.516$, which reconstructs markedly better when the coarse channel is pinned and slightly worse for edges only.

The reconstruction knobs live in the right panel's *Reconstruction* section, shown when `mz_edges` is in the layer's rack. By default (`Live` off) no reconstruction row computes on its own: each shows a reconstruction already cached for the current knobs (the recon row also its cached preview), else the data, and reads "press Run" until Run computes the row on show. *Run* computes the reconstruction either to convergence (the relative change falls under the tolerance, the iteration cap is reached, or the constraint residual rises three iterations in a row, which returns the best iterate) or for a fixed count, with progress and Stop. With `Live` on, showing the recon row computes a one-iteration preview, Decay κ, Clip and Coarse recompute it as they change, and Run continues from it. Each recon row reads its iterations, why the run stopped and its SNR against the data (mean removed; for edges only, against the data minus the synthesised coarse part). ROI results do not reconstruct yet (their rows say so), and the thumbnail shows in the 2-D view. Under *printed* the reconstruction is the research port's alternating projections with its own `Mode`, run for the iteration count when a row is shown.

Reconstructing from selected maxima (LastWave). The rack's filters reach the reconstruction: at every level the engine runs the chain's filters that apply on that level alone, and the Reconstruction section's level table decides what each level contributes. *all*: every maximum; *own* (the default): the maxima the filters keep there; *near s*: the level's own maxima within `near_radius` px of a maximum kept at level s (with the α check, only those whose modulus fits the source chain's decay $2^{\alpha(l-s)}$ within the tolerance, in $\log_2$); *coder s*: the kept positions of level s carrying this level's own transform values (Mallat & Zhong's coding mode, §IX); *predict s*: the kept positions and arguments of s with the modulus $M_s\, 2^{\alpha(l-s)}$; *open*: no constraints, so the alternating projections fill the level from the others. The *§IX preset* selects at level 2 and codes levels 1 and 3 from it (the paper codes with $J = 3$). A source must have a selection of its own (*all*, *own* or *near*). With Per-level filters on, the filter knobs edit the level the Transport shows and each level keeps its own settings (turning it on starts every level from the current ones). The canvas draws, at the level on show, the maxima the reconstruction reads there, and the section reads how many of the level's maxima those are. α comes from the analysis: each maximum propagates up the levels to the nearest maximum of the same sign within 2^{l-1} px, α is the slope of $\log_2$ of the stored modulus over the linked levels from 2 up (the coarsest level takes the α of the chains reaching it), and a line takes the median of its points'. From level 2 up the stored modulus follows $2^{l\alpha}$ (a step 0, a line -1, a point -2); level 1's staggered two-tap gradient departs from it by a factor that depends on the full direction of the edge's normal (0.86 to 1.37 on straight binary steps), so a prediction at level 1 carries that factor. In mirror mode a selection is applied as removals to all four images of the mirrored field. Every reconstruction row is keyed on the filters and the table, so each selection caches on its own.

Parameter	Default	Range	Meaning
`n_levels` (Levels J)	4	$[1, 10]$	dyadic levels; scales $2, 4, \ldots, 2^J$ px
`algorithm`	lastwave	2 choices	the authors' algorithm, or the paper's printed algorithm (research port)
`border`	mirror	mirror / periodic	the algorithm on the mirrored field, or LastWave's periodic mode
`colocate_l1` (Co-locate level 1)	off	bool	detect the finest level on co-located gradient components

`dither` (printed)	off	bool	$\pm\frac{1}{2}$ LSB dither before analysis
`interpolate` (printed)	off	bool	subpixel refinement of each maximum along its gradient
`show` (*view*)	edges	8 outputs	the raster shown in place of the data (the Outputs rows)
`recon_live` (Live, *view*)	off	bool	off: reconstruction rows wait for Run; on: they compute on their own
`kappa` (Decay κ, *view*)	1.0	$[0.1, 10]$	decay of the correction between maxima, $e^{-\kappa/2^l}$ per sample; 1 = the paper, 3.516 = LastWave
`clip` (*view*)	off	bool	clip the modulus to decrease away from each maximum (LastWave's clipping)
`run_mode` (*view*)	converge	2 choices	run to convergence, or a fixed number of iterations
`iterations` (*view*)	20	$[1, 500]$	the total count (fixed) or the cap (converge), the preview's iteration included
`tolerance` (*view*)	10^{-3}	$[10^{-6}, 10^{-1}]$	relative change of the image that counts as converged
`coarse` (*view*)	full	full / thumbnail	the coarse channel a reconstruction pins: S_{2^J}, or the one decoded from its thumbnail
`mode` (printed, *view*)	separable	3 modes	how the research port's projection interpolates between the maxima
`recon_levels` (Levels, *view*)	(every level own)	table	each level's state and source, and with Per-level filters each level's filter settings
`per_level` (Per-level filters, *view*)	off	bool	the filter knobs edit the level the Transport shows
`near_radius` (*view*)	1	$[0, 8]$ px	how far a *near* maximum may sit from a kept source maximum
`alpha_check` (α check, *view*)	off	bool	a *near* maximum must also fit its source chain's decay
`alpha_tol` (α tolerance, *view*)	0.5	$[0.05, 4]$	the $\log_2$ misfit the α check allows
`alpha_fallback` (*view*)	0	$[-3, 2]$	the α of a chain too short to fit

6.4 Wavelet skeleton · `wavelet_skeleton` — medial-axis (modulus-minima) extractor

Tang–You 2003 (the constructed compact-support wavelet κ: support radius = scale, maxima-pair separation = scale, width- and gray-invariant) + You et al. 2006 (modulus *minima* sit on the medial axis, scale-independently). A detector, not statistics: directional minima gated by a flanking ridge *pair* (each side within reach $\sim s$ must clear `edge_frac` of the peak modulus), valley contrast (`t_frac` of the flanking ridge — the paper's threshold T made local), and pair dominance (a pair under `pair_dom` of the strongest reachable ridge is someone else's slope). Border and nodata neighborhoods (one kernel support) are distrusted. On `input=gradient` the detector runs over $\|\nabla s\|$, where a diffused scarp (an *edge* in the signal) becomes a ribbon whose axis is the fault trace. Output is single-scale extrema (H-lines via `line_id`).

Parameter	Default	Range	Meaning
`s1` (s_1)	6	$[2, 64]$ px	stage-1 scale = kernel support radius; shoul cover the ribbon width
`s2` (s_2)	6	$[2, 64]$ px	refinement-stage scale
`t_frac` (T/ridge)	0.5	$[0.1, 0.9]$	keep minima below this fraction of their ow flanking ridge
`edge_frac` (Edge/peak)	0.1	$[0.01, 0.6]$	flanking-pair significance vs the *peak* modul lower to admit faint features

`n_stages`	2	$[1, 5]$	WT passes (1 = detection only; each furthe pass re-runs on the skeleton image)
`input`	signal	signal / gradient	ribbons in the signal, or scarps via $\|\nabla s\|$
`wavelet`	tang_you	tang_you / gaussian / bspline2	the papers' own comparison set; theorems h for κ only
`pair_dom` (Pair dom.)	0.5	$[0, 0.9]$	pair-dominance gate; 0 disables

6.5 CDF edges · `cdf_edges` — complex cross-diffusion edges

One complex cross-diffusion evolution (Gilboa small-θ; Barbeiro 2023 for the scale-space reading), snapshotted at the dyadic schedule $n = \sigma^2/(2\cos\theta)/\mathrm{d}t$: the Re channel evolves as the Gaussian-smoothed image and $\mathrm{Im}/\theta \approx t \cdot \mathrm{LoG}$ (machine-pinned), so one evolution *is* a Mexican-hat scale space. The nonlinear variant modulates diffusivity by the Im channel (Perona–Malik-style edge stopping through k). Final Re (`filtered`) and Im (`edge_channel`) rasters ride the result. Scales are effective Gaussian σ in px.

Edge readouts (*Edges*, `edge_mode`), each read from every snapshot:

- **Canny (Re)**, stored as `mz`: the gradient-modulus maxima of the Re channel. The gradient is the central difference; its direction is snapped to the nearest of the eight neighbour directions, and a pixel is kept where its modulus is at least that of both neighbours along it (neighbours wrap at the field's edges). This is Canny's non-maximum suppression with a single floor, $|\nabla\mathrm{Re}| > \texttt{floor} \cdot \max|\nabla\mathrm{Re}|$, and no hysteresis: one threshold and no edge tracking. The Re channel is already the Gaussian-smoothed image, so the smoothing is the diffusion's own. The same reading is the Mallat–Zhong edge definition, which makes `cdf_edges` directly comparable to `mz_edges` at matched scales; applying it to the diffusion is this app's own graft.
- **Marr (Im)**, stored as `marr`: the zero crossings of the Im channel ($\approx t \cdot \mathrm{LoG}$, the Marr–Hildreth edges), where a pixel and the next one along its row or column differ in sign, kept where the slope of Im exceeds `floor` times its maximum. This is the CDF authors' own edge concept: Barbeiro's v "in the role of edge detector".

Under the `follow` detector both readouts give way to xsmurf's contour following on the Re channel, and `edge_mode` and `interpolate` are inert.

Parameter	Default	Range	Meaning
`n_levels` (Levels J)	4	$[1, 8]$	dyadic sigmas $1, 2, \ldots, 2^{J-1}$ px; cost is quadratic in the top σ
`variant`	linear	linear / nonlinear	LCDF, or NCDF with edge-stopping
`k_edge` (k)	1.0	$[0.001, 1000]$	Perona–Malik edge threshold, image units (nonlinear only)
`theta` (θ)	0.1047	$[0.001, 0.5]$ rad	complex-diffusion angle (small-θ regime; $\pi/30$ default)
`dt`	0.2	$[0.001, 0.24]$	explicit-Euler step (stability-capped)
`edge_mode` (Edges)	mz	mz / marr	the edge readout: Canny (Re), stored as `mz`, or Marr (Im), stored as `marr` (above)
`detector`	nms	nms / follow	maxima detector
`show` (*view*)	edges	edges / filtered / edge_channel / re_scale / im_scale	the edge stack, the final Re or Im, or the Re or Im at the shown level (the Outputs rows; the *edges* row hides the maxima over any raster)
`interpolate`	off	bool	sub-pixel refinement of the maxima
`floor`	0.05	$[0, 0.5]$	detection floor (fraction of the per-snapshot maximum)

6.6 Perona–Malik edges · pm_edges — anisotropic diffusion edges

One Perona–Malik (1990) evolution — real, gradient-driven conduction $g(|\nabla I|)$, $\lambda \leq \frac{1}{4}$, adiabatic borders — snapshotted at the same nominal dyadic σ schedule as `cdf_edges`, with gradient-NMS maxima per snapshot. What it adds: past the flux peak $\phi(s) = s\,g(s)$ the diffusion runs locally *backward*, sharpening edges while the discrete maximum principle keeps it stable, so edges stay sharp and in place across the stack. Scales are nominal effective σ (the $K \to \infty$ heat-equation limit); `filtered` (the final diffused raster) rides the result.

Parameter	Default	Range	Meaning
`n_levels` (Levels J)	4	$[1, 8]$	dyadic nominal sigmas
`k_edge` (K)	1.0	$[0.001, 1000]$	edge threshold in image units: contrasts above K sharpen, below it blur
`g`	exp	exp / frac	conduction function (the paper's two)
`lam` (λ)	0.2	$[0.001, 0.25]$	explicit step (stability needs $\lambda \leq \frac{1}{4}$)
`show` (*view*)	edges	3 outputs	canvas shows the edge stack, the diffused raster (`filtered`), or the diffused raster at the shown level (`filtered_scale`); the Outputs rows, and the *edges* row hides the maxima over either
`detector`	nms	nms / follow	maxima detector
`interpolate`	off	bool	sub-pixel refinement of the maxima
`floor`	0.05	$[0, 0.5]$	detection floor (fraction of the per-snapshot maximum)

7 Decompositions

Standalone transforms heading their own chain, displaying through the ordinary raster path. They are primary analyzers: dropped onto a layer headed by another analyzer, each forks a sibling layer (§2). Every output is computed once and cached; which one is on screen is a *view* choice. To keep an output as data of its own, fork it (§1).

Group (`tucker_HOOI_HOSVD`, `ssa2d`; *view*): which components form the reconstruction — `all` (the whole kept reconstruction, exactly), or a list such as `1-3, 5`. **recon** shows the sum of the group; **residual** shows the data minus that sum (data = recon + residual of the whole kept reconstruction). So removing just the first component is Group `1` with Show = residual. The layer row names it: `recon [1-3, 5] (88%)`, `data − [1] (42%)` (the group's share). An unreadable group shows the whole reconstruction and says so. The Components window picks the group by clicking thumbnails.

7.1 PCA · pca — principal components of a multi-band field

Principal component analysis over the bands of a multi-component field ($n_y, n_x, n_c \geq 2$) (covariance-trick PCA); shows one component as a raster.

Parameter	Default	Range	Meaning
`n_components` (Components)	6	$[1, 64]$	components computed
`component` (Show PC; *view*)	1	$[1, 64]$	the component shown
`standardize`	off	bool	scale each band to unit variance first (correlation PCA)

7.2 Tucker (HOOI / HOSVD) · tucker_HOOI_HOSVD — delay-embedded Tucker decomposition (Tucker-SSA)

A Tucker decomposition of a delay embedding of the field — Broomhead–King singular-spectrum analysis carried to 2-D and multiband data by a Tucker decomposition instead of one SVD. (It was first named `tucker_havok`;

HAVOK's regression on the delay coordinates was never part of it.) Nothing is ever materialized: bases come from Grams of contiguous window slabs, and the reconstruction averages each pixel over the windows covering it.

- **Embedding.** `delay_2d` (default): *both* axes delayed — every $L \times L$ patch (L = Delays), tensor (delay$_y$, delay$_x$, y', x' [, band]), so no image axis plays "time". `delay`: a 1-D Hankel embedding along one axis (Tape axis) — for data where one axis really is time (e.g. a reconstructed tape); on an ordinary 2-D field it biases the result toward that axis. `none`: the array's own modes (rows, columns[, band]).
- **Method** — a two-state button. **HOSVD**: one pass, each mode's leading singular vectors. **HOOI**: that HOSVD refined by *HOOI sweeps* alternating passes, each mode re-derived with the others fixed (monotone in the kept core energy; the fast path equals a materialized HOOI to 10^{-8}).
- **Components** (at most 32, largest core-energy share first): under `delay_2d` one per kept (vertical, horizontal) delay-pattern pair (a, b); *Orientation* `combined` adds each pair to its twin (b, a) ($r(r+1)/2$ components for delay rank r). Tape: one per delay pattern; none: one per rows-mode vector. They sum to the reconstruction.
- Separable by construction: a pattern is a vertical pattern $\otimes$ a horizontal one, so an oblique structure spreads over several pairs. `ssa2d` is the rotation-invariant alternative.

Parameter	Default	Range	Meaning
`embed` (Embedding)	delay_2d	delay_2d / delay / none	see above
`axis` (Tape axis)	rows	rows / cols	the time axis of the 1-D tape (greyed out unless `delay`)
`n_delays` (Delays)	32	$[2, 512]$	embedding dimension L (patch $L \times L$ under delay_2d); the lag between delays is 1 px
`rank_delay` (Rank: delay)	4	$[0, 512]$	delay patterns kept (both delay modes under delay_2d); 0 = full
`rank_time`, `rank_space`	0	$[0, 10^5]$	ranks of the rows/cols (y'/x') modes; 0 = full
`rank_band` (Rank: band)	0	$[0, 64]$	rank of the band mode (multiband only); 0 = full
`method` (Method)	HOSVD	HOSVD / HOOI	the algorithm (a toggle button)
`sweeps` (HOOI sweeps)	2	$[1, 16]$	HOOI refinement passes (greyed out under HOSVD)
`show` (*view*)	recon	recon / residual / component	what the canvas shows
`component` (*view*)	1	$[1, 64]$	the component shown (Show = component)
`pairs` (Orientation; *view*)	separate	separate / combined	component numbering under delay_2d; also what a Group refers to
`group` (Group; *view*)	all	text	the components the recon sums (above)
`cluster` (*view*)	1	$[1, 64]$	the cluster shown (largest share first)
`cluster_distance` (*view*)	0.5	$[0.05, 0.95]$	where the clustering tree is cut

Superseded: `tucker_havok` stays registered (browser Dev group) so saved projects open and resolve exactly as before; its single *HOOI sweeps* knob chose the algorithm (0 = HOSVD).

7.3 2-D SSA · `ssa2d` — 2D singular-spectrum analysis

Golyandina & Usevich (2010). Every $L_r \times L_c$ window of the image is one column of the Hankel-block-Hankel matrix $\mathbf{W}$ ($L_rL_c \times K_rK_c$, $K = N - L + 1$); *one* SVD $\mathbf{W} = \sum_i \sqrt{\lambda_i}\, U_i V_i^{\mathsf{T}}$ gives the eigentriples. Reshaped, U_i is an **eigenarray** — an $L_r \times L_c$ pattern of any shape, not restricted to separable products, which makes 2D-SSA invariant to rotations of the image axes — and V_i a **factor array** ($K_r \times K_c$). Each rank-1 term, averaged back onto the image (every pixel over the windows covering it), is an **elementary component**; all of them sum to the image. Ranks (the paper's §4, pinned by tests): an oblique cosine $\cos 2\pi(\omega_x k + \omega_y l)$ is rank 2, a product of cosines rank 4, a sum of m 2-D exponentials rank m.

Solvers. None forms $\mathbf{W}$.

- *Lanczos* (the default), *PROPACK* and *randomized* use FFT products: $\mathbf{W}^{\mathsf{T}}u$ and $\mathbf{W}v$ are 2-D correlations of the image, done at its own size, whose kept parts never wrap (Korobeynikov 2010; Golyandina et al. 2015). They find only the k leading eigentriples — ARPACK's Lanczos on the implicit $\mathbf{W}\mathbf{W}^{\mathsf{T}}$; PROPACK's bidiagonalization of $\mathbf{W}$, with full reorthogonalization (its default partial one returns ghost copies of the leading triplet when the 32-bit FFT products are adjoint only to $\sim 10^{-7}$); or a randomized range finder whose power iterations are re-orthonormalized after every product (Halko et al. 2011; Lopes et al. 2024). Components are FFT convolutions of eigenarray and factor array. They agree with the dense route to float32 tolerance. Measured on a 2048^2 field with a 64×64 window and $k = 16$: Lanczos 6 s, randomized 25 s, PROPACK 42 s.
- *Dense*: the lag covariance $\mathbf{W}\mathbf{W}^{\mathsf{T}}$ ($(L_rL_c)^2$) accumulated from blocks of window rows, its full eigenbasis, then $\mathbf{W}^{\mathsf{T}}U$ in the same blocks; float64 BLAS, equal to an explicit SVD of $\mathbf{W}$ to 10^{-9}. Limited to windows of 1600 entries (40×40, about 4 s on a 256^2 field). An FFT solver hands over to it when the kept count reaches the window size.

The shares $\lambda_i / \operatorname{tr}(\mathbf{W}\mathbf{W}^{\mathsf{T}})$ are exact on every route (the trace is the sum of the squared pixels weighted by their window counts). Each eigenarray is signed so that its largest entry is positive. Large outputs are stored at the app's precision. Single-band fields only. NaN pixels are filled with the finite mean for the decomposition and re-masked in every output. Also returned: the **w-correlation** matrix (the paper's Def. 3.18) of the kept components — the grouping aid shown in the Components window.

Keeping and grouping (Lopes et al. 2024). *Keep: energy* treats `n_components` as the most to find and keeps the fewest whose cumulative share reaches the threshold (the row note says when it was not reached). This 2D-SSA is uncentred, so on a raw field the first eigentriple, the mean, often carries most of the energy on its own. *Show: cluster* shows one group of an automatic grouping: average-linkage clustering on $1 - |\text{w-correlation}|$, cut at the cluster distance, clusters ordered by share. The cluster knobs are view settings, so changing them never re-decomposes.

Edges. A pixel near the edge of the analysed region is covered by fewer windows. On a region of interest the tool reads a margin of real data around it (`margin_windows` windows of $\max(L_r, L_c) - 1$ pixels; at one window every ROI pixel is fully covered) and crops back. On the whole field there is no data beyond the edge. Lopes et al.'s mirror extension is available (*Whole-field edges: mirror*), but off by default: in 2-D a reflected wave is a different wave (an oblique wave flips a frequency component), so the extension adds rank. On two test waves it raised the edge error 3–18×, where the real-data margin of an ROI lowered it by about 10%.

Parameter	Default	Range	Meaning
`rows_window`, `cols_window`	16	$[2, 512]$	window $L_r \times L_c$ ($L_rL_c \le 1600$ for dense)
`n_components` (Components kept)	16	$[1, 64]$	leading eigentriples kept (the most, with Keep: energy)
`keep`	count	count / energy	keep `n_components`, or the fewest reaching the energy threshold
`energy_threshold` (Energy)	0.9	$[0.5, 0.9999]$	cumulative share to reach (Keep: energy only)

`solver`	lanczos	lanczos / propack / randomized / dense	the eigentriple route (above)
`power_iterations`	2	$[0, 10]$	randomized solver only
`margin_windows` (Edge margin)	1	$[0, 4]$	ROI margin in windows; also the mirror width
`edge_extension` (Whole-field edges)	none	none / mirror	mirror-extend the whole field (off: it adds rank in 2-D)
`show` (*view*)	recon	recon / residual / component / cluster	what the canvas shows
`component` (*view*)	1	$[1, 64]$	the elementary component shown
`group` (Group; *view*)	all	text	the components the recon sums (above)
`cluster` (*view*)	1	$[1, 64]$	the cluster shown (largest share first)
`cluster_distance` (*view*)	0.5	$[0.05, 0.95]$	where the clustering tree is cut

7.4 DWT · `dwt2d` — the periodized orthogonal pyramid

The field is mirror-extended once in each axis and rounded up to a multiple of 2^J, then decomposed with an orthogonal bank (symlets, Daubechies, coiflets, or `bl3`, the Battle–Lemarié order-3 bank of Pottier, Turiel & Garçon 2008) in periodization mode, the one mode whose synthesis is the analysis's adjoint. Every band is a per-level row reconstructed alone onto the native grid: the `approximation` at level j (the low-pass at scale 2^j px) and the `horizontal`, `vertical` and `diagonal` details; the details of every level plus the coarsest approximation sum back to the field exactly. Cycle spinning averages each row over random shifts (translation invariance at the price of that many transforms). Non-finite pixels are filled with the finite mean for the transform and restored on every row. The Transport walks the levels; the pyramid is the foundation of the wavelet-leader tools.

Parameter	Default	Range	Meaning
`wavelet`	sym7	the orthogonal banks	the pyramid's bank
`level`	0	$[0, 10]$	levels; 0 = the full depth $\lfloor \log_2 \min(2n_y, 2n_x) \rfloor - 1$, at most 8
`cycle_spin`	off		average every row over random shifts
`n_spins`	8	$[1, 64]$	the shifts averaged
`show` (*view*)	coarsest	coarsest / the four bands	the coarsest approximation, or the row drawn at the Transport's level

8 Inpainting and fusion

One engine, three routes (Pottier, Turiel & Garçon 2008 for the cascade, the compressed-sensing notebooks for the cycle-spun sparsity). The unknown lives on the mirror-extended working image; the data terms are the target under its weights and every guide on its own grid (a shift on the same grid, a block average on a nested one, a warp on a foreign one — a guide may be resampled inside its measurement, the target never); the prior is the cycle-spun soft threshold of the detail bands (*sparsity*, FISTA with restarts), the paper's parent-from-children inference over the abacus of $\ln|\eta|$ blended by ρ (*cascade*, the paper's plain loop), or the first then the second (*both*). The known pixels are corrected after every step through the universal kernel and put back exactly at the end. Held-out blocks of clear pixels measure the error of each route; the $\ln|\eta|$ and $\ln|c|$ distributions inside the regions against the clear ones say whether the fill kept the cascade (the Histograms window).

8.1 Inpaint · `inpaint` — fill declared regions

Regions: the non-finite pixels (always), a threshold on a band of the stack (connected components over `mask_min_px`), polygons drawn in the Mask mode (stored in file pixels), or a sent mask raster; each region is a *gap* (every scale

inferred) or a *veil* (shadow, haze: the fine-scale geometry is data under a gain or an offset, the amplitude and the coarse levels are inferred; experimental). Sends (the Sends dialog): a *value* prior (the same quantity: another day, another satellite, a climatology), a *structure* guide (another band, or a DEM shaded under the scene sun), *weights* (a coherence or error raster, for the target or for a send), a *mask*, a *veil seed* DEM. A foreign value or structure send closes the cascade routes. Result: the filled band (`raster_out`, equal to the data on known pixels), `fill_mask`, the rows `residual` and `heldout`, and the stamps `_route`, `_grid_relations`, `_seed`, `_iterations`, `_returned`, `_stop`, `_variance_error`, `_heldout`, `_eta_distance`, `_amp_distance`, `_terms_dropped`, `_veil`, `_fill_kind`.

Parameter	Default	Range	Meaning
`route`	sparsity	sparsity / cascade / both	the prior
`wavelet`	sym7	the orthogonal banks	the pyramid's bank
`level`	0	$[0, 10]$	0 = the route's depth (sparsity: maximum, at most 8; cascade: t depth)
`lam_rel`	0.2	$[0, 10]$	λ relative to $\hat{\sigma}\sqrt{2 \ln N}$ (sparsity,
`n_iter`	0	$[0, 5000]$	0 = the route's default (200, 20)
`tol`	10^{-3}	$[0, 1]$	sparsity stop: relative change ins regions
`stop_var_err`	0.01	$[0, 1]$	cascade stop: variance error on k
`seed`	0	$[0, 2^{31}]$	cycle-spin shifts, the held-out dra synthesis pass
`batch`	1	$[0.05, 1]$	the mini-batch of known pixels p
`cascade_synthesis`	off		the coarse-to-fine synthesis pass paper)
`target_band`		text	the band id on a stack (empty =
`mask_source`	nodata	nodata / threshold / region / send / all	where the regions come from
`mask_band`, `mask_op`, `mask_value`, `mask_min_px`, `threshold_kind`			the threshold: band id (empty = $>$ or $<$, the value, the smallest c gap or veil
`region_kind`	gap	gap / veil	the kind of a sent mask's regions
`veil_model`, `veil_levels`, `veil_passes`	gain, 3, 2		the veil: gain (positive data) or (log-domain), its fine levels J_v, t passes
`sun_azimuth`, `sun_elevation`	0, 45	degrees	the scene sun when the loader st (ASTER does)
`prior_weight`	0.3	$[0, 10]$	λ_v of a value prior
`structure_weight`, `structure_levels`	0.1, 3		λ_s and J_s of a structure guide
`abacus`, `abacus_file`, `abacus_min_pairs`	self, , 1000		the $\ln \lvert\eta\rvert$ histograms: this field's a corpus file
`correct`	on		the known-pixel correction after
`heldout_blocks`, `heldout_side`	8, 16		the held-out blocks (0 = none) a in px
`show` (*view*)	filled	filled / residual / heldout	the raster on the product layer

9 Microcanonical MFA

Per-pixel singularity exponents $h(x)$ from the model $T(x, r) \approx \alpha(x)\, r^{h(x)}$ (Turiel 2008), $D(h)$ by direct histogram — the formalism that is *not* canonical/Legendre. One tool per functional; both expose the Pont 2006 estimator pair: **regression** (per-pixel log–log slope over the scale grid; the prefactor cancels exactly; resolves both limbs of $D(h)$) and **punctual** (the finest scale alone, ensemble-mean normalized; sharpest localization, the most-singular-set extractor; cannot resolve the right limb; carries an $\alpha(x)/\langle\alpha\rangle$ bias). Both estimators share the same cross-scale no-support mask — they agree on *where* an exponent exists and differ in *how*. Under punctual, κ/`n_scales` shape

only the mask and `r2_map` is honestly all-NaN.

9.1 Microcanonical MFA, measure · `holder_measure` — gradient-measure route

T = positive unit-mass kernel over $\|\nabla s\|$ (finite differences first, then g_0-class smoothing). For measure-like / edge-sparse fields; slope = h. The scale knob r_1 is the kernel's *dilation scale* (for the Gaussian: its standard deviation) — positive kernels have no zero crossings, and their resolution floor is the pixel, except for the compactly supported q-Gaussian (below). Kernel trade-off (Turiel App. A): Gaussian resolves every h; Lorentzian truncates at $h < 2\beta - d$ but localizes sharpest; q-Gaussian interpolates; `frac_gaussian` fractionalizes the envelope, $e^{-\rho^n/2}$ (n=2 is the Gaussian exactly).

The q-Gaussian is Tsallis's $e_q^{-\rho^2/2r^2}$ with $e_q^x = [1 + (1-q)x]_+^{1/(1-q)}$ (Borges et al. 2004, eq. 9), for $-1 \le q \le 3$. Above $q = 1$ it has power-law tails (Student-t / Lorentzian: $q \equiv 1 + 1/\beta$ with ρ^2/r^2 rescaled); $q = 1$ is the Gaussian; below $q = 1$ it is *compactly supported*, radius $r\sqrt{2/(1-q)}$ — $q = 0$ is the Epanechnikov paraboloid, $q = -1$ a half-dome of radius r, and $q \to -\infty$ the Dirac δ. For $q \ge 2$ the 2-D kernel has no finite mass (the Lorentzian $\beta \le 1$ case), so the result depends on the field size; the q knob shows a warning there. The compact kernel (below $q = 1$) is the steepest decrease there is, so it carries a resolution floor: the r_1 knob warns when its support disc, of radius $r/\sqrt{\beta(1-q)}$ for width β, is less than 2 px across, i.e. below $r_1 = \sqrt{\beta(1-q)}$ (at most 1 px at the default width).

9.2 Microcanonical MFA, multiaffine · `holder_multiaffine` — wavelet-projection route

$T = |\psi_r * s|$ with a zero-mean wavelet applied to the *raw signal* (derivative order lives inside the kernel). For smooth / function-class fields (DEMs); slope = γ, with $h_{\text{measure}} = \gamma - 1$. Here r_1 is the *zero-crossing radius*: every wavelet in the menu is calibrated to change sign at radius r, so for a wavelet with one crossing the fig.-2 minimum-resolution rule reads $r_1 \ge 1$ (more crossings, or a compact support, raise it: *Resolution floor* below). The g-ladder is the vanishing-moment axis (order n measures only $\gamma < n$): `g1`/`g3` are the fractional family at n=1, 3; `g2` *is* the Ricker; `lorentzian_marr` is the Laplacian (Marr operator) of the Lorentzian envelope (true tail $\rho^{-2\beta-2}$, truncation $\gamma \ge 2\beta$); `frac_gaussian` makes the order a real knob ($\psi \propto {}_1F_1(\frac{n+2}{2}; 1; -\rho^2/2\sigma^2)$, n=2 reproduces the Ricker exactly): n is a fractional *differentiation* order ($\hat{\psi} \sim |k|^n$ at low frequency), so exact moments vanish only for the integer orders below n — a fractional n is not "n vanishing moments" — and $\gamma < n$ is measurable.

`q_mexican` is the q-Mexican hat of Borges et al. (2004, eq. 15): the second derivative of the q-Gaussian *raised to the power* $2 - q$. In 2-D (the Laplacian) this gives

$$\psi_q(\rho) \;\propto\; \left[1 - (2-q)\, b\rho^2\right] \left[e_q^{-b\rho^2}\right]^q, \qquad b = \frac{1}{(2-q)\, r^2},$$

so the zero crossing sits at $\rho = r$ for every q, and $q = 1$ is the Ricker exactly. The paper's $q < 3$ is the 1-D bound: in 2-D the Laplacian of $[e_q]^{2-q}$ vanishes identically at $q = 2$ and never changes sign above it, so the knob runs $-1 \le q \le 1.95$. For $q \ge 0$ the wavelet is bounded; for $q < 0$ it diverges at its cutoff (still integrable) and the knob shows a warning. Because $[e_q]^{2-q}$ is itself a q'-Gaussian with $q' = 1/(2-q)$, this is the family of Laplacians of q-Gaussians under a relabelled index: `q_mexican` at q equals `lorentzian_marr` at $\beta = (2-q)/(q-1)$ for $1 < q < 2$ (e.g. $q = 1.5 \equiv \beta = 1$), with tail $\rho^{-2/(q-1)}$ and truncation $\gamma \ge 2(2-q)/(q-1)$.

The q width β (both q-kernels) is the β of $e_q^{-\beta x^2}$, a toggle away from the paper's two choices.

- **Fixed** (default $\beta = \frac{1}{2}$): the unit-variance Gaussian per component at $q = 1$, in any dimension. The scale r keeps its meaning at $q = 1$: $\sigma = r$ for the q-Gaussian, the zero crossing at r for the q-Mexican hat. At other q the q-Mexican hat's zero crossing drifts to $r/\sqrt{2\beta(2-q)}$.
- **q-paired**: Borges et al.'s $\beta = 1/(3-q)$ holds the q-Gaussian's *escort* (q-)variance $\int x^2[e_q]^q / \int [e_q]^q$ at one. In d dimensions that variance is $1/(\beta(d + 2 - dq))$ per component, so the pairing is $\beta = 1/(d + 2 - dq)$: $1/(3-q)$ in 1-D and $\beta(q) = 1/(2(2-q))$ here.
- The q-Mexican hat's zero crossing sits exactly at that escort RMS radius, so the pairing also pins it (at r, for every q: the tool's classic calibration). The two settings agree at $q = 1$.

- The pairing exists exactly where the 2-D q-Mexican hat is admissible, $q < 2$: the escort variance and the zero mean both need $d + 2 - dq > 0$. A q-paired q-Gaussian therefore refuses $q \geq 2$.

- In real space the normalisation is done numerically at every scale, so the paper's closed-form constants are not used. (In 2-D they happen to be elementary: $A_q = \sqrt{\beta(3-q)(1+q)/\pi}$ for the q-Mexican hat, $B_q = \sqrt{\beta(3-q)/\pi}$ for the modulated q-Gaussian. The flat Jacobian $\rho\, d\rho = du/2\beta$ turns every integral into a Beta function with integer arguments, where 1-D keeps $\Gamma(\frac{1}{2})$-type ratios.)

Resolution floor on r_1. A discretized wavelet must separate its positive and negative parts, so its zero crossings set the minimum attainable resolution, and more crossings cost resolution (Turiel et al. 2008, fig. 2 and §4.2.1); Turiel (2009) puts a Mexican hat's inner minimum scale at "several pixels". Neither gives a number. The tools read the rule as: every sign lobe at least 2 px across at r_1, the central disc counted by its diameter. The edge of a compact support is not a sign change, but it closes the last lobe. The r_1 knob warns below the floor and never refuses.

- One crossing at r (`g1`, `g2`, `lorentzian_marr`, `frac_gaussian` for $n \leq 2$): the floor is $r_1 = 1$, as before.

- More crossings: `g3` 1.32 px (its ring is $1.52\,r$ wide); `frac_gaussian` at $n = 4$ and 5, 1.41 and 1.46 px.

- `q_mexican`: its crossing sits at $\rho_0 = r/\sqrt{2\beta(2-q)}$ (r when q-paired) and, below $q = 1$, its support ends at $\rho_0\sqrt{(2-q)/(1-q)}$. The negative ring between them narrows as q falls, and below $q \approx 0.875$ (q-paired) it is the narrowest lobe: floor 2.73 px at $q = 0.5$ and 4.42 px at $q = 0.1$. A fixed width β is a pure dilation of the wavelet, so it rescales the floor by $\sqrt{2\beta(2-q)}$.

- `holder_measure`: only the compact q-Gaussian has a floor (above).

Wavelet space: real or Fourier (both tools). *Real* (the default, Turiel's construction) samples each kernel on the pixel grid and normalizes it there. *Fourier* evaluates the same continuous kernel's closed-form 2-D transform at $r\mathbf{k}$: a Matérn function of k for a q-Gaussian with $q > 1$ (and the Lorentzian, $q = 1 + 1/\beta$), a Jahnke–Emde function for $q < 1$, $k^n e^{-k^2/4u_0}$ for the fractional family, evaluated stably at every q. The kernel is then band-limited rather than sampled.

- Measure route: unit mass exactly. Multiaffine route: zero mean exactly, and the kernel is the L^1-dilated continuous wavelet, so T equals the real construction's times the mother wavelet's L^1 norm (a constant that cancels in every slope) wherever both are exact.

- They differ where sampling matters: at the smallest scales, for compact kernels (the Fourier one rings slightly), and for heavy tails, which real space cuts at the edge of the padded grid (about 10^{-4} at the default padding for a $\rho^{-6.7}$ tail).

- Fourier is refused on the measure route for `frac_gaussian` (no closed-form transform), and for a q-Gaussian at $q \geq 2$ or a Lorentzian with $\beta \leq 1$ (no finite mass to normalize by).

The same r_1 value is not numerically comparable across the two tools, and γ values are not comparable across wavelets.

Parameter	Default	Range	Meaning (both holder tools)
`estimator`	regression	regression / punctual	Pont estimator pair (see above)
`r_min` (r_1)	1.0	$[0.1, 32]$ px	finest scale (dilation scale / zero-crossing radius per tool); warns below the resolution floor
`kappa` ($\kappa = r_2/r_1$)	8	$[1.1, 200]$	scale-range width; the knob that changes the physics
`n_scales`	6	$[2, 64]$	samples in the range (nearly irrelevant vs κ; keep small)
`wavelet`	(per tool)	method-true menu	see tool text
`q_tsallis` (Tsallis q, q_T)	1.5	$[-1, 3]$ / $[-1, 1.95]$	q-family shape, measure / multiaffine (q_gaussian / q_mexican only); warns at measure $q \geq 2$, multiaffine $q < 0$

`beta` (β)	1.0	$[0.1, 8]$	Lorentzian tail exponent (Lorentzian family only)
`frac_n` (n)	2.0	$[0.05, 8]$	fractional order (frac_gaussian only)
`q_pairing` (q width)	fixed	fixed / q-paired	the q-kernel's width β: fixed, or $\beta(q) = 1/(2(2-q))$ (below); greyed out unless q_gaussian / q_mexican
`q_beta` (q β)	0.5	$[0.05, 5]$	the fixed width of $e_q^{-\beta\rho^2/r^2}$ (greyed out while q-paired)
`construction` (Wavelet space)	real	real / fourier	sample each kernel on the grid, or build it from its closed-form transform (above)

9.3 Band reconstruction: the `band` and `band_mask` outputs

Both Microcanonical MFA tools reconstruct the field from a singularity band. The band $h \in [h_{\mathrm{lo}}, h_{\mathrm{hi}})$ of the cached h map, cleared of islands outside the size range, is the mask; the reconstruction is the gradient-restricted Fourier inversion of the analysed values on that mask (Turiel eq. 65, symmetric-extension border fix). Two output rows carry them, *band reconstruction* (`band`) and *band mask* (`band_mask`), and `Show` picks what the canvas draws: the h map, the band or its mask. The band row reads its PSNR against the data and the share of pixels in the band, the mask row the share.

The band knobs are view-only and sit in the right panel's *Reconstruction* section. Only the two outputs read them: scrubbing the band recomputes the reconstruction from the cached h map, and the analysis key leaves them out. A saved project gains them at their defaults. Sweeping the band and watching the reconstruction is the workflow. *Reconstruct...* (the Band dialog) previews the band live; on a whole-field layer headed by a Microcanonical MFA tool, a band row of a stack included, it writes the band into that step's knobs and shows the band row. On an ROI layer, and on a layer headed by `holder_map`, it adds a `band_recon_measure` or `band_recon_multiaffine` child, which runs the same $h(x)$ engine with the band as keyed knobs and shows the reconstruction as its raster.

Parameter	Default	Range	Meaning (band knobs; engine knobs above)
`show` (*view*)	h_map	3 outputs	what the canvas draws: `h_map`, or the `band` reconstruction or its `band_mask` (the Outputs rows)
`h_lo` / `h_hi` (*view*)	-2 / 0	$[-6, 6]$	the band $h_{\mathrm{lo}} \le h < h_{\mathrm{hi}}$, in the estimator's own frame
`min_island` / `max_island` (*view*)	0 / 0	px	drop mask components outside this size range before inversion
`connectivity` (*view*)	8	8 / 4	island connectivity (8 keeps diagonal ribbons whole)

Superseded: `band_recon_measure` and `band_recon_multiaffine` stay registered in the browser's Dev group, so a project that holds one opens and resolves as before; at the same knobs their reconstruction equals the `band` output bit for bit. `holder_map` and `band_recon` (the pre-split conflated tools, estimator × wavelet crossed) remain registered for saved projects and live in the Dev group too; their physics is identical to the split tools at the old defaults.

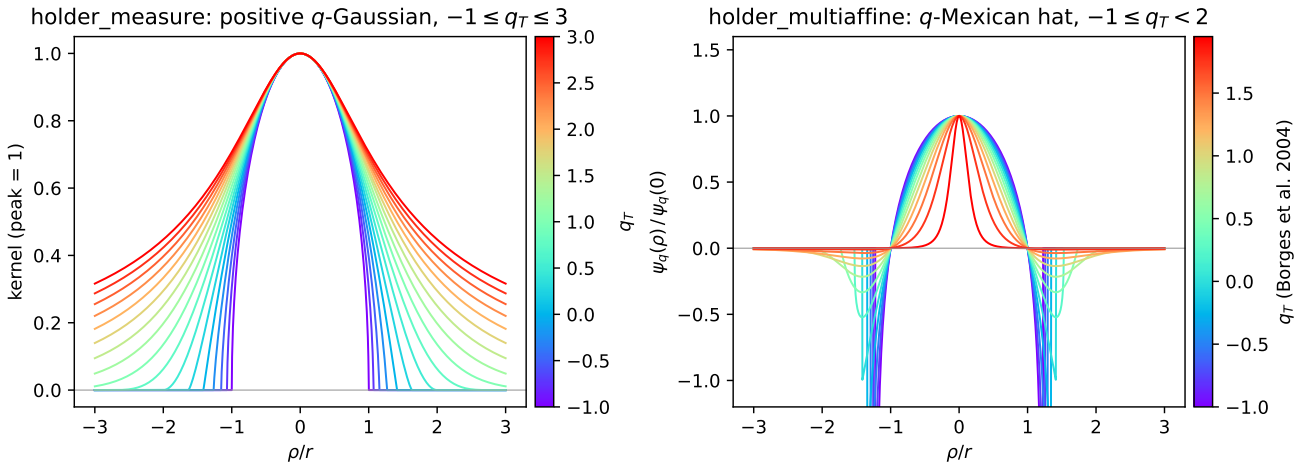


Figure 2: The q family. Left: `holder_measure`'s positive q-Gaussian, $-1 \le q \le 3$ (compact below $q = 1$). Right: `holder_multiaffine`'s q-Mexican hat (Borges et al. 2004, in 2-D), $-1 \le q < 2$; every curve crosses zero at $\rho = r$, and for $q < 0$ the negative lobe diverges at the cutoff. Profiles scaled to 1 at the centre.

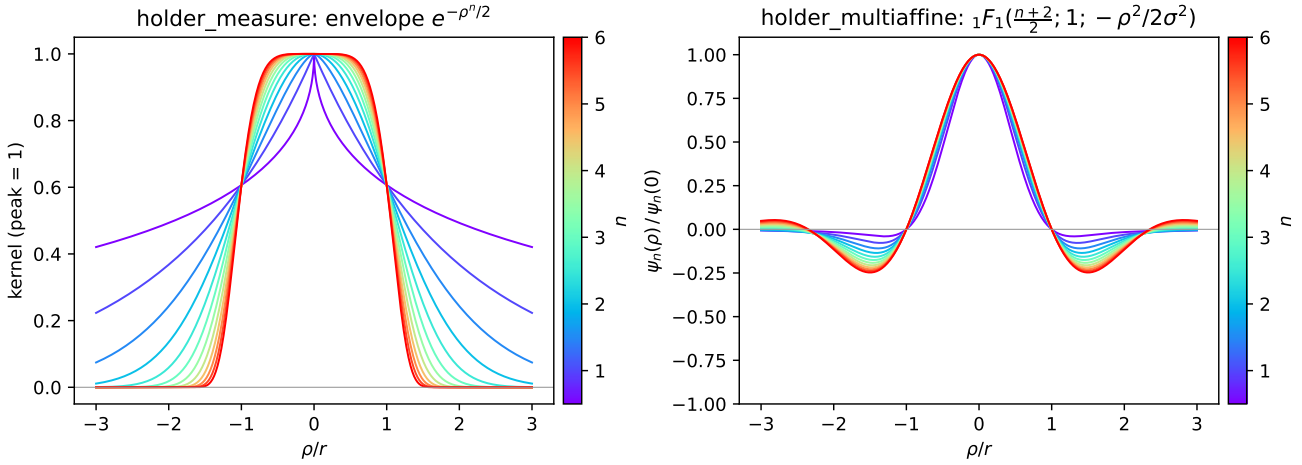


Figure 3: The fractional Gaussians. Left: `holder_measure`'s envelope $e^{-\rho^n/2}$ ($n < 2$ fattens the tail, $n > 2$ flattens the top; every curve passes $e^{-1/2}$ at $\rho = r$). Right: `holder_multiaffine`'s order-n wavelet ${}_1F_1(\frac{n+2}{2}; 1; -\rho^2/2\sigma^2)$, σ set per order so the first zero crossing sits at $\rho = r$.

10 Topology

10.1 Chain topology · `chain_topology` (Transform)

Builds the incidence graph over the WTMM maxima-line skeleton (no parameters). It applies after an analysis that carries an extrema stack and V-chains (the WTMM tools), passes that analysis through and adds the topology, which downstream topology filters query.

10.2 Min V-chains per H-line · `min_vchains` (Filter)

Keeps H-line points that at least `min_vchains` maxima lines (V-chains) thread. It applies after Chain topology. Default UI opens at 1 (show everything computed; turn up to prune).

11 Selection: extrema

Act on the extrema of the level the Transport shows (§1), after any tool that produces an extrema stack (the applicability table, §2). In mode `sup_all_scales`, `hline_modulus` also reads the V-chains threading each line, and `hline_holder` reads the per-point α the Mallat–Zhong analysis stamps.

Device.param	Default	Range	Meaning
`orientation_wedge.centre`	0	deg	keep extrema whose gradient strike f wedge
`orientation_wedge.half_width`	90	$[0, 90]$ deg	wedge half-width (90 = no-op)
`orientation_wedge.north`	grid	grid/true/magnetic	the NorthRef of `centre`; frame error multi-degree errors
`modulus_threshold.frac`	0	$[0, 1]$	per-point: keep $\|W\| \geq$ frac $\times$ scale p
`hline_length.min_len/max_len`	1 / 0	points	per-line point count range; kills orph confetti and stitching seams; -1 orp
`hline_modulus.frac`	0	$[0, 1]$	per-*line* modulus keep-fraction
`hline_modulus.mode`	sup	sup / sup_all_scales / mean / segments	line statistic: peak at this scale; peak threading chains (needs chains); mea per-segment between modulus minim
`hline_holder.min_alpha/max_alpha`	-3 / 3	$[-3, 3]$	keep lines whose Hölder α lies in the (`mz_edges`: the per-line α of its analy layer without α passes)
`hline_holder.keep_unfitted`	on	bool	keep points whose chain could not be

12 Selection: V-chains

Act on cross-scale chains, the WTMM maxima lines (V-chains). They apply after an analysis that produces V-chains (the applicability table, §2); the other analyzers stamp `chains: []`, and their cross-scale linking is reserved work.

Device.param	Default	Range	Meaning
`chain_holder.cutoff`	-3	$[-3, 3]$	keep chains with Hölder $\geq$ cutoff
`chain_holder.estimator`	ols	ols / max	per-chain h: log–log OLS slope, or the max-modulus scaling read
`chain_modulus.threshold`	-99	$[-99, 32]$	keep chains whose max $\log_2 \|W\| \geq$ threshold
`chain_length.min_len`	2	$[2, 64]$	minimum scales spanned

12.1 Seam classifier · `chain_classify`

Classifies chains by their Hölder exponent and geometry into step-like ($h \approx 0$: fault-scarp class) vs delta-like ($h \approx -1$: thin-ridge class) lineaments, with a straightness gate; flagged or excluded per `action`, excluded chains optionally drawn as ghosts.

Parameter	Default	Range	Meaning
`estimator`	ols	ols / max	per-chain h estimator
`fit_a_min`	2.0	$[0.01, 64]$	fit floor: scales below this are excluded from the h fit
`h_step_lo` / `h_step_hi`	-0.15 / 0.15	$[-3, 3]$	the step-class h window
`h_delta_lo` / `h_delta_hi`	-1.2 / -0.8	$[-3, 3]$	the delta-class h window
`straightness_min`	0.85	$[0, 1]$	end-to-end / arc-length straightness gate
`action`	flag	flag / exclude	tag matches, or drop non-matches
`show_ghosts`	on	bool	draw excluded chains desaturated

13 Groups, points, Dev

`group_paint` / **Group filter** · `group_filter`: the arrangement view's commit transaction — `group_paint` stamps committed group tags onto chain copies (`spec_json`/`signature` are written by the commit flow, not hand-edited); `group_filter` keeps or drops one named group.

Back-project points · `backproject`: registers a point catalogue onto a raster layer's grid (the point-layer transform). `target` names the raster layer; the `_target_*` params are the resolved grid geometry, written by the shell — not hand-edited.

`derived_vectors`: a derivative dataset's frozen extrema and maxima lines, read from its own file (§1). Placed by the shell (browser Dev group); its one parameter (the file) is hidden.

`stub_wavelet` / `stub_holder` / `stub_wedge`: parameter-schema test fixtures (the browser's Dev group); no analysis role. They declare no contracts, so every step after them applies.

Superseded tools stay registered in the Dev group, so a saved project that names one opens and resolves: `holder_map` and `band_recon` (the pre-split Hölder tools), `band_recon_measure` and `band_recon_multiaffine` (§9.3), `tucker_havok`, and `scale_select` (§1).

14 Planned: q-generalized wavelets

Not implemented yet — recorded here so the design stays in one place. Borges et al. (2004) generalize the Mexican hat (above), the Morlet wavelet (a q-Gaussian-modulated plane wave, their eq. 23) and the trigonometric functions (an even/odd pair of q-trigonometric wavelets, their §4), and note that "other wavelets, as well as scaling functions, may be generalized along the lines here introduced, for instance, the Meyer wavelet or the sinc function."

- **q-generalized `g1` and `g3` for `wtmm2d`.** WTMM's analyzing wavelets must stay *derivatives of a smoothing function* ($\psi = \partial\theta$), so that maxima mark edges. The filters are built in the Fourier domain, so they need the *2-D* transform of the q-Gaussian, a Hankel transform with Bessel order $1/(q-1) - d/2$, one half lower than the paper's 1-D order. That transform is now built and evaluated stably at every q (the Matérn and Jahnke–Emde functions used by *Wavelet space: Fourier*, §9).
  - q-`g1`: $\partial\theta_q$ with θ_q the unit-mass q-Gaussian, so the L^1 normalization is unchanged ($\hat{\theta}_q(0) = 1$); it exists for $q < 2$.
  - q-`g3`: the power-$(2-q)$ recipe makes the Laplacian's envelope a q'-Gaussian with $q' = 1/(2-q)$, whose mass is finite only for $q < 3/2$. For $3/2 \le q < 2$ the wavelet is still finite (k^2 cancels the divergence at $k = 0$), but $\hat{\theta}(0) = 1$ can no longer fix its scale, so it needs another normalization there, such as unit L^1 norm.
- **q-Morlet** and the **q-trigonometric** pair (oriented/directional analysis).
- **q-Meyer** and **q-sinc**, as the paper suggests.

References

- A. Arnéodo, N. Decoster and S. G. Roux (2000), A wavelet-based method for multifractal image analysis. I. Methodology and test applications on isotropic and anisotropic random rough surfaces, *Eur. Phys. J. B* 15, 567–600.
- E. P. Borges, C. Tsallis, J. G. V. Miranda and R. F. S. Andrade (2004), Mother wavelet functions generalized through q-exponentials, *J. Phys. A: Math. Gen.* 37, 9125–9137. doi:10.1088/0305-4470/37/39/006
- A. Turiel, H. Yahia and C. J. Pérez-Vicente (2008), Microcanonical multifractal formalism — a geometrical approach to multifractal systems: Part I. Singularity analysis, *J. Phys. A: Math. Theor.* 41, 015501.
- A. Turiel (2009), Relevance of multifractal textures in static images, book chapter.

- M. Unser and T. Blu (1999), Construction of fractional spline wavelet bases, *Proc. SPIE* 3813 (Wavelet Applications in Signal and Image Processing VII), 422–431.
- M. Unser and T. Blu (2000), Fractional splines and wavelets, *SIAM Review* 42, 43–67.
- S. Mallat and S. Zhong (1992), Characterization of signals from multiscale edges, *IEEE Trans. PAMI* 14, 710–732.
- P. Perona and J. Malik (1990), Scale-space and edge detection using anisotropic diffusion, *IEEE Trans. PAMI* 12, 629–639.
- N. Golyandina and K. Usevich (2010), 2D-extension of singular spectrum analysis: algorithm and elements of theory, in *Matrix Methods: Theory, Algorithms and Applications* (V. Olshevsky, E. Tyrtyshnikov, eds.), World Scientific, 449–473.
- A. Korobeynikov (2010), Computation- and space-efficient implementation of SSA, *Statistics and Its Interface* 3, 257–268.
- N. Golyandina, A. Korobeynikov, A. Shlemov and K. Usevich (2015), Multivariate and 2D extensions of singular spectrum analysis with the Rssa package, *J. Stat. Software* 67(2), 1–78.
- N. Halko, P.-G. Martinsson and J. A. Tropp (2011), Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions, *SIAM Review* 53, 217–288.
- F. Lopes, D. Gibert, V. Courtillot, J.-L. Le Mouël and J.-B. Boulé (2024), On the optimization of singular spectrum analyses: a pragmatic approach, preprint.
- D. S. Broomhead and G. P. King (1986), Extracting qualitative dynamics from experimental data, *Physica D* 20, 217–236.
- L. De Lathauwer, B. De Moor and J. Vandewalle (2000), A multilinear singular value decomposition, *SIAM J. Matrix Anal. Appl.* 21, 1253–1278; and On the best rank-1 and rank-$(R_1, \ldots, R_N)$ approximation of higher-order tensors, *ibid.*, 1324–1342 (HOSVD and HOOI).

Appendix: result contracts

- The named data contracts with the result keys each covers, and what each tool consumes and produces, are tabled in §2.
- **Extrema schema** (all edge/skeleton analyzers): per-scale {`x, y, mod, arg, line_id`}; `line_id` groups H-lines, -1 marks isolated points. `scales` is the matching scale array (px).
- **h-map contract** (holder tools): `h_map` displays in place of the field through the ordinary raster path; `r2_map` rides along (all-NaN under punctual). The `band` and `band_mask` outputs are lazy rasters with keys of their own (§9.3). `raster_out` (`band_recon_measure`, `band_recon_multiaffine`) displays the reconstruction; `h_map`/`band_mask` ride for the dialogs.
- **cdf extras**: `filtered` (final Re) and `edge_channel` (final Im) rasters.
- The vector (3-D) view draws one extrema layer: the level the layer shows (§1).
- **Decomposition extras**: `ssa2d` — `ssa_recon`, `ssa_residual`, `ssa_components` (n, n_y, n_x), `ssa_eigen_share`, `ssa_eigenarrays` (n, L_r, L_c), `ssa_factor_arrays` (n, K_r, K_c), `ssa_w_correlation` (n, n). `tucker_HOOI_HOSVD` — `tucker_recon`, `tucker_residual`, `tucker_components` / `tucker_combined_components` with their energies, core, factors, energy per mode; on a multiband field also `tucker_recon_bands` / `tucker_residual_bands` (n_y, n_x, n_c) (the display shows band 1). A view step writes `raster_out` and, for a named view, `_view_note` (what the layer row appends).
- **Inpainting extras**: `inpaint` — `fill_mask` (region ids), `_estimate` (the estimate before the final step, float32), `_heldout_mask`, `_hists` (the per-level $\ln|\eta|$ and $\ln|c|$ histograms), `_abacus`, `_warnings`,

_filled_fraction; dwt2d — scales (2^j px), _dwt_levels. A fill clone's provenance carries fill; an analyzer result after it carries _fill.

- **Derivative file**: the RasterField npz keys (schema, name, values, x_axis, y_axis, units, frame_meta, provenance) plus, when it carries vectors, vec_e_* (extrema, CSR-packed), vec_c_* (maxima lines) and vec_scales. A vector-only derivative's raster is all-NaN on the same grid.